Mapping game coordinates to the minimap

Players move in world units; the minimap is a picture in pixels. A wrong scale or a missing axis flip still draws a believable map, with every event in the wrong place. This is how the conversion works and how it was checked.

1
Conversion function
in the pipeline; the browser gets map positions only

72,676
Path points converted
0 outside their map's square

0.07
Worst round trip, world units
0.29 of a pixel; all from JSON rounding

16 + 39
Coordinate tests passing
pipeline + browser, 25 Oct 2026



AmbroseValley · every recorded path



GrandRift · every recorded path



Lockdown · every recorded path

Humans blue, bots amber, drawn from the shipped match files on the shipped minimaps. Numbers computed by `pipeline/tools/explainer_pdfs.py`.

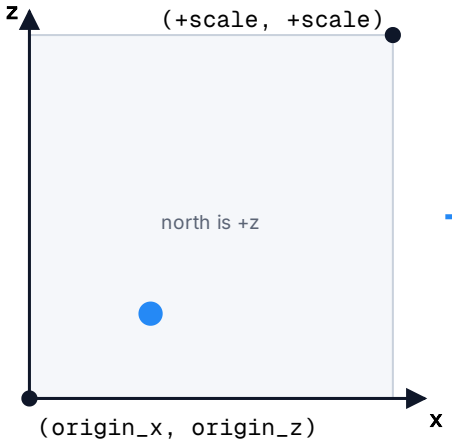
THE PROBLEM

Four coordinate systems, two of them upside down to each other

The game logs x, y, z in world units with z pointing north. Images count rows from the top. The trick is to go through a neutral 0 to 1 space in between, and to do it in exactly one place.

1 · World

game units, from the parquet

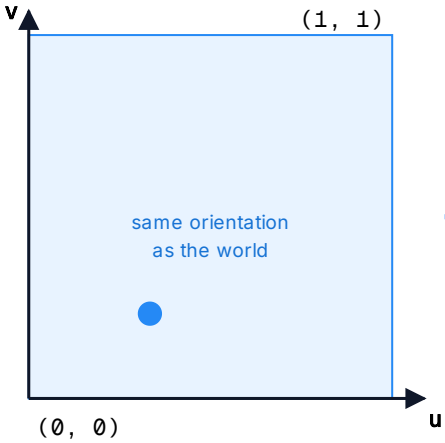


y is height: ignored for a 2D map

pipeline

2 · Map position (u, v)

0 to 1, stored in the JSON

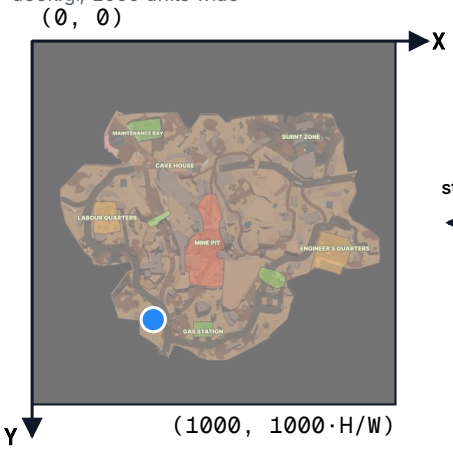


no image size involved

browser

3 · Drawing space

deck.gl, 1000 units wide



Y grows downward, like image rows

stretched over

4 · Minimap image

pixels, W × H



row 0 is the top

BitmapLayer bounds
`[0, 1000·H/W, 1000, 0]`

```
# 1 → 2 · projection.py
u = (x - origin_x) / scale
v = (z - origin_z) / scale
```

```
# 2 → 3 · space.ts
X = u × 1000
Y = (1 - v) × 1000 × H / W
```

```
# 2 → pixel (for checks and figures)
px = u × W
py = (1 - v) × H
```

The **1 − v** is the flip: world z grows north (up the page) while image rows and the drawing space grow down. The **H / W** factor keeps a non-square image in proportion.

CALIBRATION

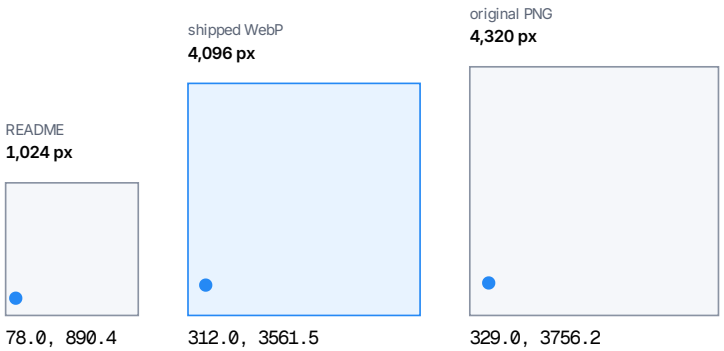
Each map is a world square laid over its image

The data README gives a scale and an origin per map: the world square from the origin to origin + scale on both axes covers the whole image. Its formula is right. Its image size is not: it says 1024 px, and none of the three minimaps is.

MAP	SCALE	ORIGIN X	ORIGIN Z	WORLD X	WORLD Z	ORIGINAL IMAGE	SHIPPED WEBP	WORLD UNITS PER PX, X / Z
AmbroseValley	900	-370	-473	-370 to 530	-473 to 427	4,320 × 4,320	4,096 × 4,096	0.2197 / 0.2197
GrandRift	581	-290	-290	-290 to 291	-290 to 291	2,160 × 2,158	2,160 × 2,158	0.2690 / 0.2692
Lockdown	1,000	-500	-500	-500 to 500	-500 to 500	9,000 × 9,000	4,096 × 4,096	0.2441 / 0.2441

Why go through 0 to 1 instead of straight to pixels

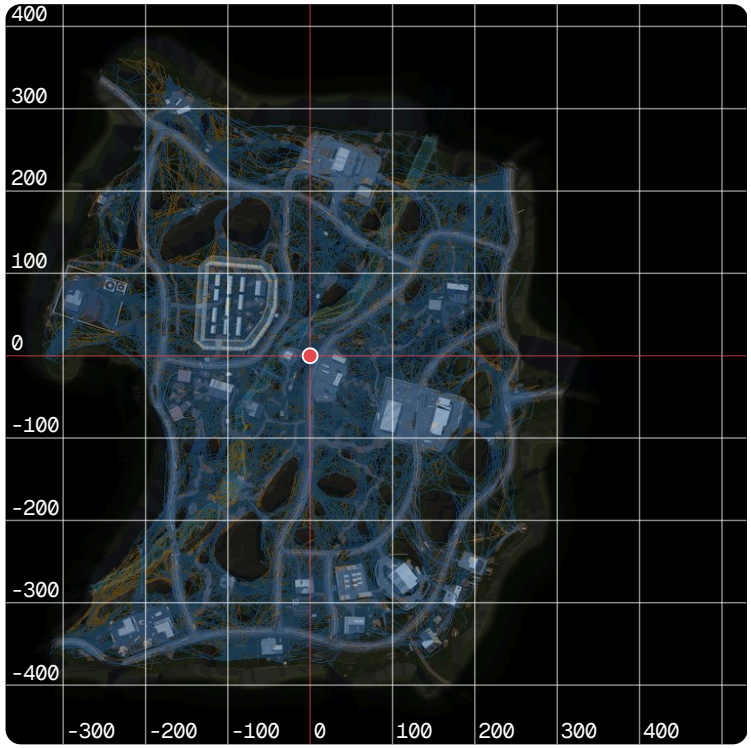
Images get resized. Lockdown's 9,000 × 9,000 px original ships as 4,096 × 4,096, and the README assumed 1024. A position stored as a share of the map is right at every size, so the pipeline stores u and v, and each consumer multiplies by its own width and height. The README's own example lands at the same spot on all three sizes:



Same u = 0.0762, v = 0.1305 every time; only the multiplier changes. The README expects (78, 890) on 1024 px.

GrandRift isn't square

Its image is 2,160 × 2,158. Treating it as square would put the south edge 2 px off, so width and height are always used separately (drawing space height 999.07).



AmbroseValley with world grid lines every 100 units (labels are world x along the bottom, world z up the side), placed with the production formulas. Red: world x = 0 and z = 0. The square runs from x −370 to 530 and z −473 to 427, so the lines don't fall on round pixel positions.

WORKED THROUGH

Two positions, step by step

First the data README's own example, which the unit tests reproduce. Then a real kill from the data, the same one followed in the data-flow PDF.

README example · AmbroseValley

```
x = -301.45, z = -355.55 · scale 900, origin (-370, -473)
u = (-301.45 − (-370)) / 900 = 68.55 / 900 = 0.0762
v = (-355.55 − (-473)) / 900 = 117.45 / 900 = 0.1305
px = 0.0762 × 1,024 = 78.0
py = (1 − 0.1305) × 1,024 = 0.8695 × 1,024 = 890.4
```

Matches the README's (78, 890). Without the flip, py = 0.1305 × 1,024 = 133.6: the point would jump from near the bottom of the map to near the top.

Real kill · Lockdown · match 00b34c64

```
x = -207.80, z = 176.78 · y = 50.38 ignored
u = (-207.80 − (-500)) / 1,000 = 0.2922
v = (176.78 − (-500)) / 1,000 = 0.6768
X = 0.2922 × 1000 = 292.2 drawing space
Y = (1 − 0.6768) × 1000 = 323.2
pixel on the 4,096 px WebP = 1197, 1324
```

The match file stores only u and v, rounded to 4 decimals. The browser never sees x or z.

The pipeline side: `pipeline/src/lila_pipeline/projection.py`

```python
def world_to_uv(map_id, x, z):
    cal = MAPS[map_id]          # scale, origin_x, origin_z
    u = (x - cal.origin_x) / cal.scale
    v = (z - cal.origin_z) / cal.scale
    return u, v                 # never clamped: callers count
                                # anything outside 0 to 1
def uv_to_pixel(u, v, width, height):
    px = u * width
    py = (1.0 - v) * height     # rows count down, z counts up
    return px, py
```

Shortened from the file. An unknown map name raises an error instead of guessing.

The browser side: `web/src/map/space.ts`

```
const SPACE_WIDTH = 1000
spaceHeight(map) = 1000 * map.height / map.width

uvToSpace(u, v)   = [u * 1000, (1 - v) * spaceHeight]
imageBounds(map)  = [0, spaceHeight, 1000, 0]
                    # the minimap's BitmapLayer
uvToWorld(u, v)   = { x: u * scale + origin_x,
                      z: v * scale + origin_z }
                    # cursor readout only
```

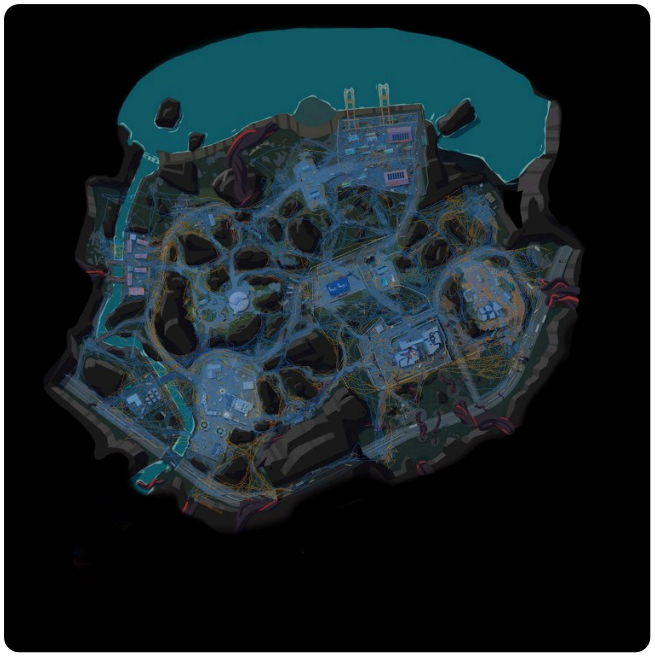
Summarised from the file. Paths, markers, heatmap pictures and hotspot outlines go through uvToSpace, and the image through imageBounds, so they share one space.

TESTING IT AGAINST THE ART

Getting it wrong on purpose shows the real mapping fits

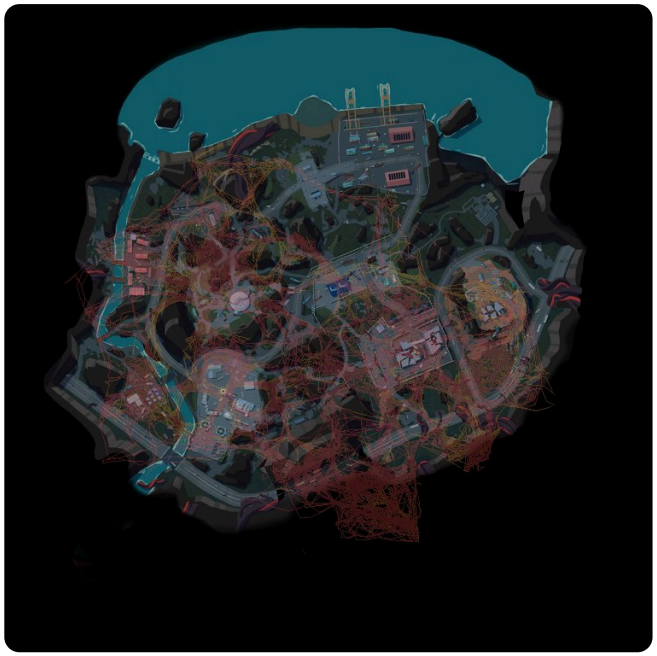
Every recorded path on Lockdown, drawn three ways on the shipped minimap. There are no surveyed landmarks to check against, so the test is whether the routes fit the roads, buildings and coastline, and whether plausible mistakes visibly don't.

Production mapping



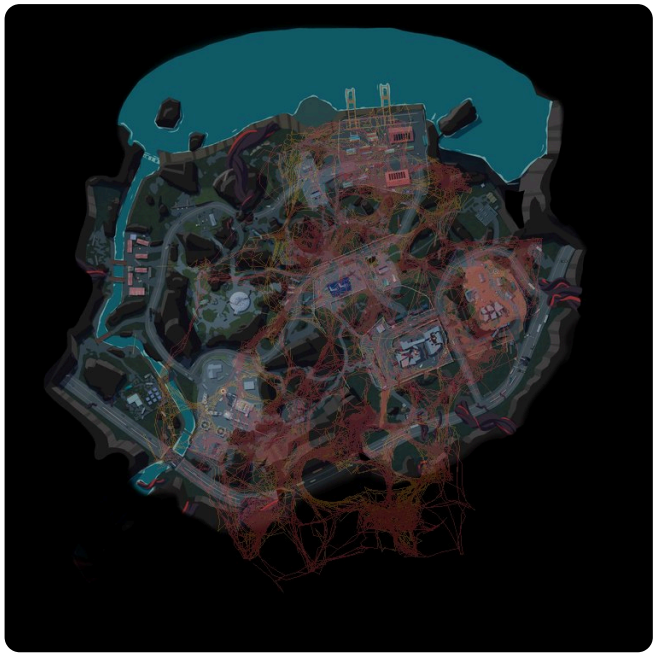
u, 1 – v. Routes follow the island's roads and stay on land; the black margin around the art stays empty.

Control: y flip removed



u, v. The routes are mirrored top to bottom: they cross the art at random and spill into the black margin.

Control: x and z swapped



v, 1 – u. The routes are rotated: same shapes, but they no longer follow roads or the coastline.

Human paths blue in the production mapping and red in the controls; bot paths amber in all three. Judged by eye, not by a pixel-colour score: dark roads and art make pixel counts unreliable, so the maths on page 7 is the pass/fail part.

TESTING IT AGAINST THE ART

The same check on AmbroseValley and GrandRift

The flip matters most near the top and bottom edges: points near the horizontal middle barely move when mirrored, so the edges are where to look.

AmbroseValley · production



AmbroseValley · no flip



GrandRift · production



GrandRift · no flip



AmbroseValley

The production routes trace the road network. Mirrored, the dense southern routes move north and off the roads.

GrandRift

Production routes cluster on the named areas printed on the map (Mine Pit, Engineer's Quarters, Gas Station, Labour Quarters). Mirrored, shared start points move into the black area beyond the map edge.

Shared start points

With the production mapping, several journeys start at the same spots on the map's edges, which fits fixed spawn points. In the controls those spots land in empty space.

VERIFICATION

The maths is the pass/fail check; the pictures are supporting evidence

Unit tests pin down the formula's behaviour; a round trip on every exported point shows the two directions agree on real data.

Unit tests: 16 pipeline, 39 browser, all passing on 25 Oct 2026

- The README's worked example gives (78, 890) on a 1024 px image.
- World corners land on the image corners and the world centre on the image centre, for all three maps.
- +x moves right, +z moves up.
- GrandRift's 2,160 × 2,158 image scales each axis separately.
- Values outside the map are reported, never clamped; an unknown map is rejected.
- The real minimap sizes are checked, and every recorded position falls inside its map's square.
- Browser: drawing space, cursor back to world, and the inverse of each function.

Where the real positions fall

MAP	POINTS	OUTSIDE	U RANGE	V RANGE
AmbroseValley	48,457	0	0.050 to 0.746	0.103 to 0.926
GrandRift	5,712	0	0.110 to 0.941	0.165 to 0.792
Lockdown	18,507	0	0.093 to 0.848	0.215 to 0.829

Players use part of each image; the rest is black margin. Nothing needed clamping.

Round trip: match file back to the parquet

MAP	POINTS CHECKED	MAX ERROR, WORLD UNITS	MEAN ERROR	LIMIT SET BY 4-DECIMAL ROUNDING	MAX ERROR, WEBP PIXELS
AmbroseValley	48,457	0.0636	0.0345	0.0636	0.29
GrandRift	5,712	0.0409	0.0223	0.0411	0.15
Lockdown	18,507	0.0707	0.0381	0.0707	0.29

Each exported path point (u, v as stored in the JSON) was turned back into world x, z and compared with its parquet row. All 72,676 points line up, and no error exceeds the rounding limit, so the formulas themselves add no error.

Shown

The pipeline implements the README formula exactly, the browser inverts it exactly, real routes fit the art, and the plausible mistakes visibly don't.

Not shown

That the README's scale and origin are what the game uses. That needs world coordinates for two or three landmarks from the level editor; until then the mapping is consistent with the maps, not proven exact.